

Terraform Task 5

Why Terraform is important

1. **Infrastructure as Code (IaC)**
Write infrastructure in .tf files → version-controlled, repeatable, and auditable.
2. **Automation & Speed**
Create servers, networks, databases, and services in minutes with init → plan → apply.
3. **Consistency & No Human Error**
Same code = same infrastructure across dev, test, and prod.
4. **Multi-Cloud Support**
Works with AWS, Azure, GCP, and on-prem from a **single tool**.
5. **State Management**
Tracks real infrastructure using a **state file**, enabling safe updates and drift detection.
6. **Idempotency**
Running Terraform multiple times won't break infra — it only changes what's needed.
7. **Easy CI/CD Integration**
Integrates smoothly with Jenkins, GitHub Actions, GitLab CI, etc.
8. **Scalable & Modular**
Reuse infrastructure using **modules** as your environment grows.
9. **Cost & Resource Control**
Plan before apply → know exactly what will be created or destroyed.
10. **Industry Standard**
Widely used in DevOps and cloud roles; strong interview and production relevance.

One-line interview answer

Terraform is important because it automates infrastructure provisioning using code, ensuring consistency, scalability, and multi-cloud support with safe, repeatable deployments.

1. Watch Terraform-05 video.
2. Execute the script shown in the video.

1.Watch the Terraform-05 video.

2.EXECUTE THE SCRIPT SHOWN IN THE VIDEO

—>

Terraform Remote state and state locking:

=====

We can store terraform configuration files and state file in github or any other repository but it is not good practise.

We use s3,terraform storage,hashicorp consul to store the state file.

State locking is used to lock the state file so that no two users can execute the state file at the same point of time.

Remote backend and state locking:

=====

We can use s3 as remote backend and dynamo db for state locking.

Create s3 using terraform:

=====

```
resource "aws_s3_bucket" "s3_bucket" {  
  bucket = "s3backend"  
  acl = "private"  
}
```

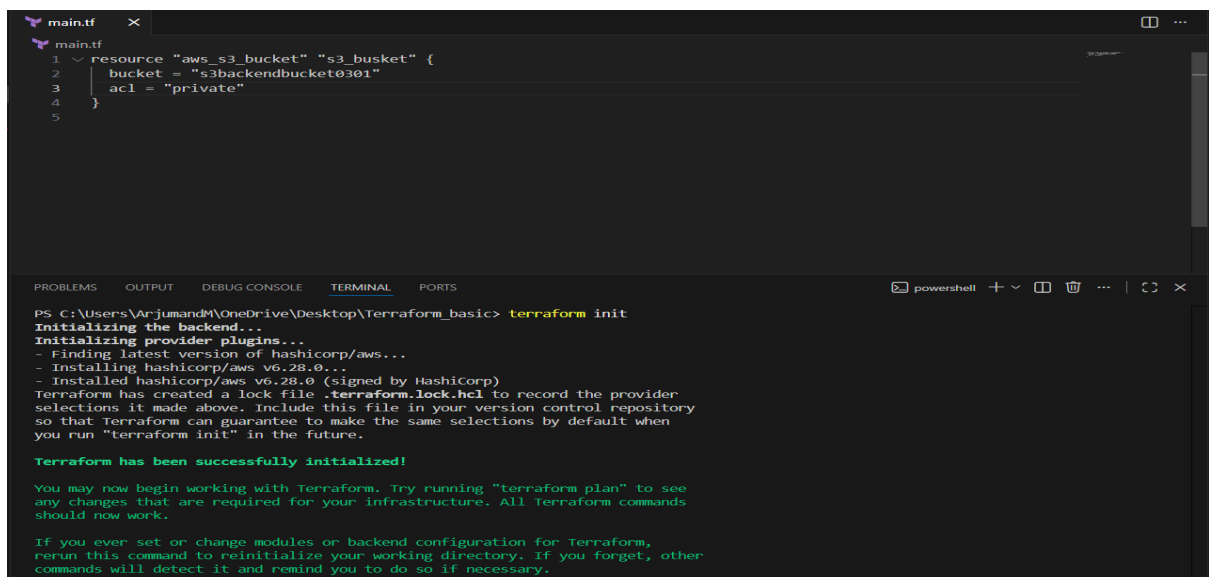
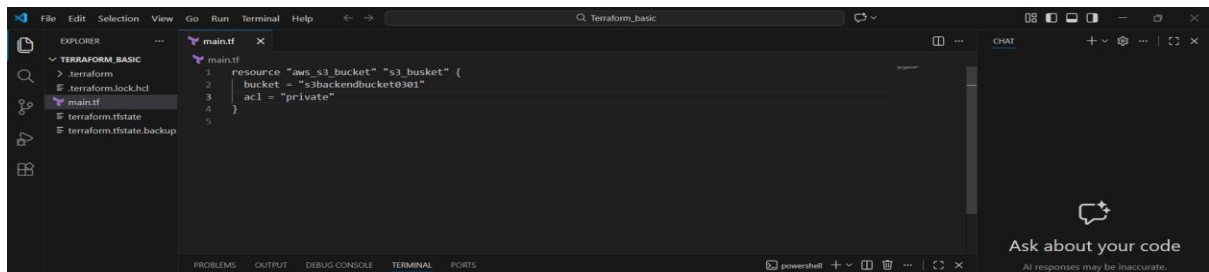
Create dynamo db using terraform:

Create dynamo db using terraform:

=====

```
resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {  
  
  name = "terraform-state-lock-dynamo"  
  
  hash_key = "LockID"  
  
  read_capacity = 20  
  
  write_capacity = 20  
  
  attribute {  
  
    name = "LockID"  
  
    type = "S"  
  
  }  
  
}
```

—-> first —> terraform init —>and then



```
acl is deprecated. Use the aws_s3_bucket_acl resource instead.

Error: creating S3 Bucket (s3backendbucket): operation error S3: CreateBucket, https response error StatusCode: 409, RequestID: 7T1TN1
XC1XW0EPwH, HostID: f2GAWQ0hhpsjIyLYK0wpvpK1UqMSWoolngg3hbcRJYa5F444z3lUXsRnPrbsduUbf9In/Vy2tFA=: BucketAlreadyExists: The requested buc
ket name is not available. The bucket namespace is shared by all users of the system. Please select a different name and try again.

with aws_s3_bucket.s3_bucket,
on main.tf line 1, in resource "aws_s3_bucket" "s3_bucket":
1: resource "aws_s3_bucket" "s3_bucket" {

PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  create
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [ ] [ ] [ ] [ ] [ ] [ ]

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

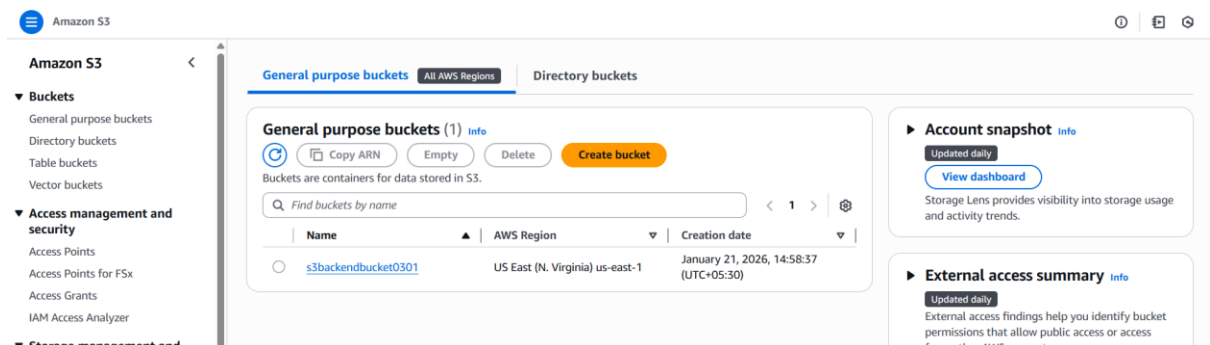
aws_s3_bucket.s3_bucket: Creating...
aws_s3_bucket.s3_bucket: Creation complete after 6s [id=s3backendbucket0301]

Warning: Argument is deprecated

with aws_s3_bucket.s3_bucket,
on main.tf line 3, in resource "aws_s3_bucket" "s3_bucket":
3:   acl = "private"

acl is deprecated. Use the aws_s3_bucket_acl resource instead.

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> [ ]
```



DynamoDB

- 1.Watch the Terraform-05 video.
- 2.EXECUTE THE SCRIPT SHOWN IN THE VIDEO

—>

Terraform Remote state and state locking:

=====

We can store terraform configuration files and state file in github or any other repository but it is not good practise.

We use s3,terraform storage,hashicorp consul to store the state file.

State locking is used to lock the state file so that no two users can execute the state file at the same point of time.

Remote backend and state locking:

=====

We can use s3 as remote backend and dynamo db for state locking.

Create s3 using terraform:

=====

```
resource "aws_s3_bucket" "s3_bucket" {  
  bucket = "s3backend"  
  acl = "private"  
}
```

Create dynamo db using terraform:

Create dynamo db using terraform:

=====

```
resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {  
  name = "terraform-state-lock-dynamo"  
  hash_key = "LockID"  
  read_capacity = 20  
  write_capacity = 20  
  attribute {  
    name = "LockID"  
    type = "S"  
  }  
}
```

—> first —> terraform init —>and then

```
5 resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {  
6   name = "Terraform-state-lock-dynamo"  
7   hash_key = "LockID"  
8   read_capacity = 20  
9   write_capacity = 20  
10  
11   attribute {  
12     name = "LockID"  
13     type = "S"  
14   }  
15 }
```

```

with aws_s3_bucket.s3_bucket,
on main.tf line 3, in resource "aws_s3_bucket" "s3_bucket":
3:   acl = "private"

acl is deprecated. Use the aws_s3_bucket_acl resource instead.

(and one more similar warning elsewhere)

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_dynamodb_table.dynamodb-terraform-state-lock: Creating...
aws_dynamodb_table.dynamodb-terraform-state-lock: Still creating... [00m10s elapsed]
aws_dynamodb_table.dynamodb-terraform-state-lock: Creation complete after 15s [id=Terraform-state-lock-dynamo]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

```

aws

Search

[Alt+] Q

United States (N. Virginia)

EC2 (0607-9593-2818)

EC2

DynamoDB

Tables

DynamoDB

Dashboard

Tables

Explore items

PartiQL editor

Backups

Exports to S3

Imports from S3

Integrations

Share your feedback on Amazon DynamoDB

Share feedback

Tables (1)

info

Find tables

Filter by tag

Any tag value

Filter by tag value

Any tag value

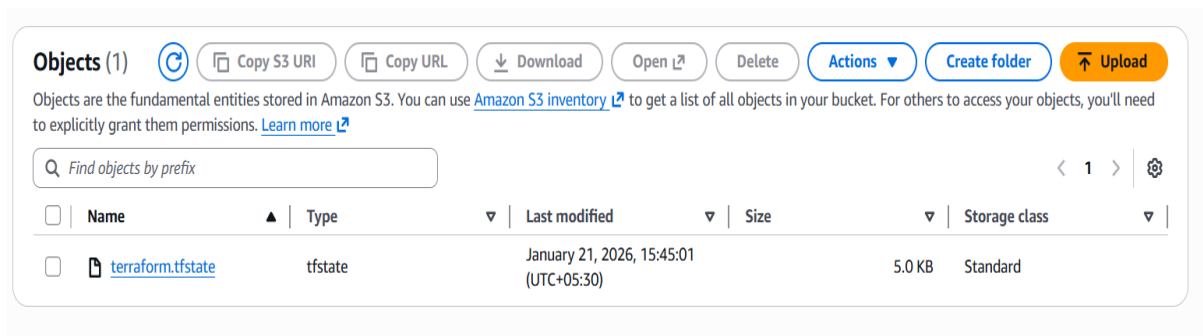
January 21, 2026, 15:34 (UTC+5:30)

Actions

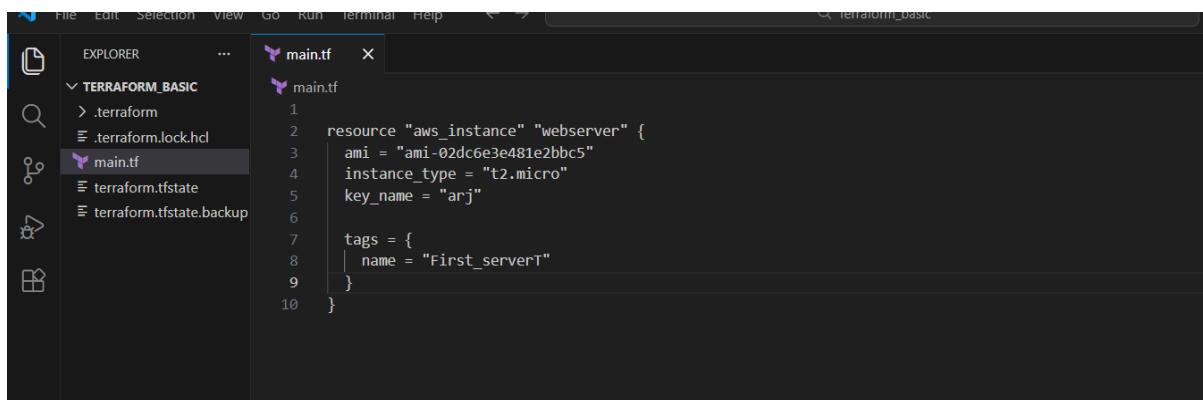
Delete

Create table

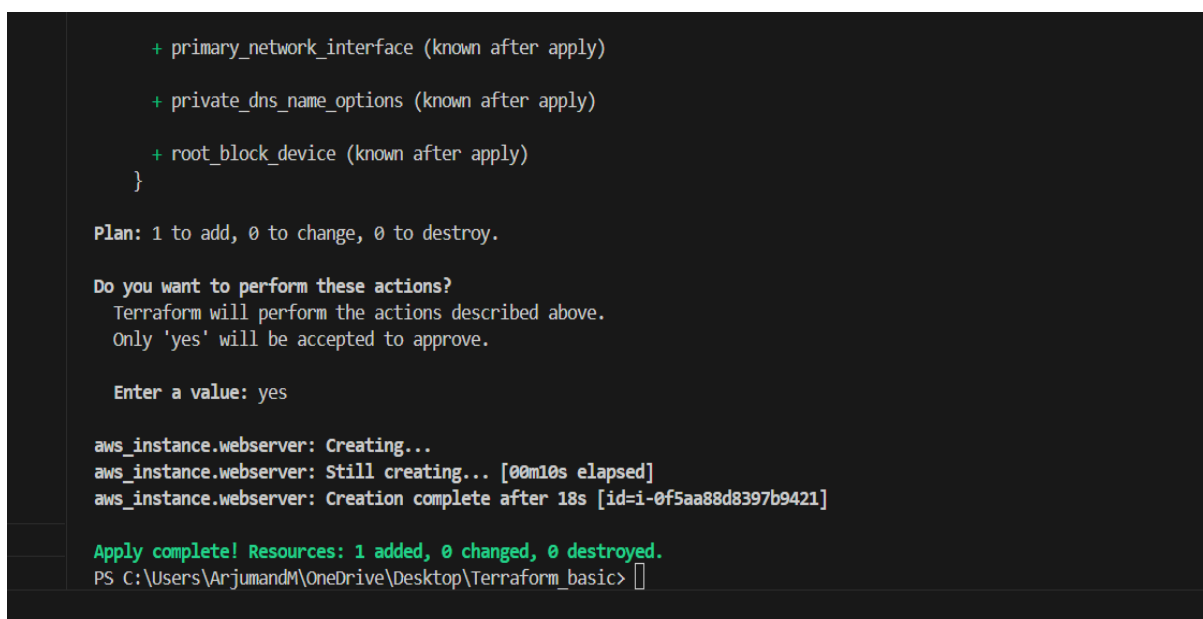
☐	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read cap
☐	Terraform-state-lock-dynamo	Active	LockID (S)	-	0	0	Off	☆	Provisioned

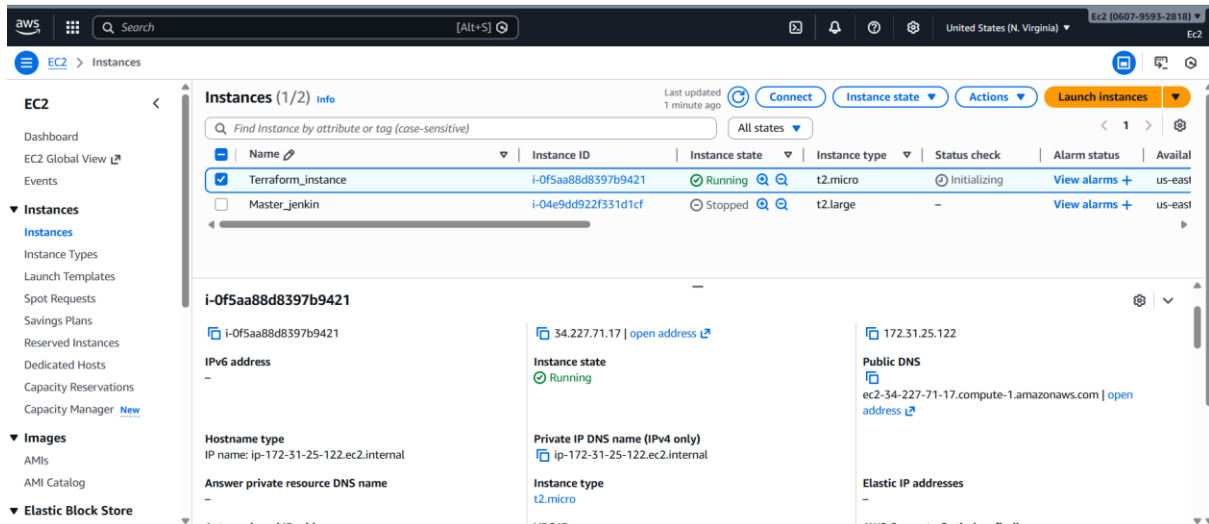


Creating instance with help of terraform



- Enter terraform init & terraform apply





```
resource "aws_instance" "test-server" {

  ami = "ami-07ff62358b87c7116"

  instance_type = "t2.micro"

  key_name = "arj"

  tags = {

    name = "First_serverT"

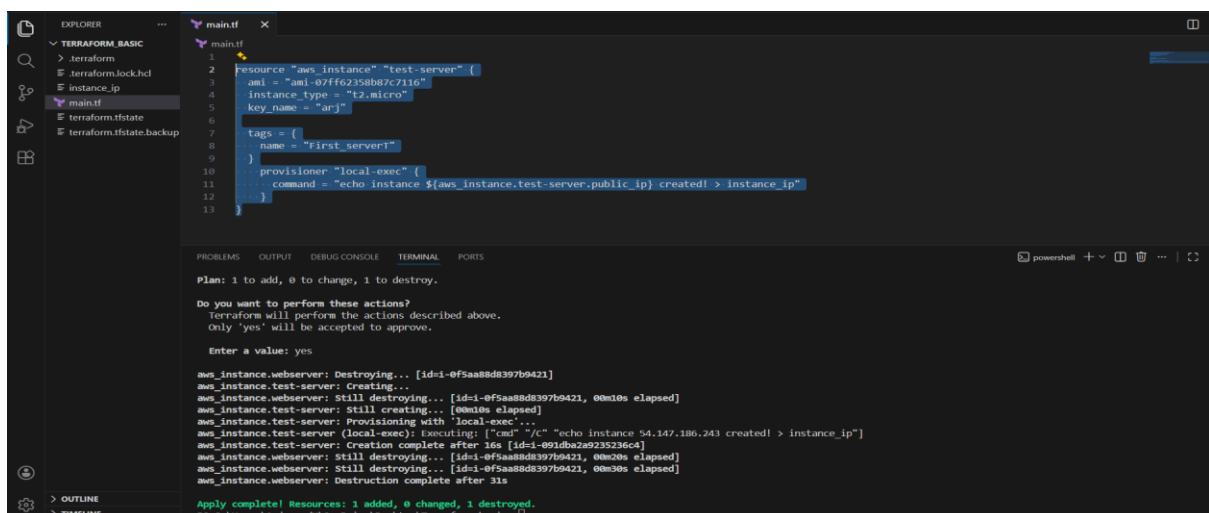
  }

  provisioner "local-exec" {

    command = "echo instance ${aws_instance.test-server.public_ip} created! > instance_ip"

  }

}
```



Find Instance by attribute or tag (case-sensitive)							
Name		Instance ID	Instance state	Instance type	Status check	Alarm status	Availability
☒	provisioner_instance	i-091dba2a9235236c4	Running	t2.micro	Initializing	View alarms +	us-east-1
☐	Terraform_instance	i-0f5aa88d8397b9421	Terminated	t2.micro	-	View alarms +	us-east-1
☐	Master_jenkin	i-04e9dd922f331d1cf	Stopped	t2.large	-	View alarms +	us-east-1

i-091dba2a9235236c4							
Details Status and alarms Monitoring Security Networking Storage Tags ▼ Instance summary info Instance ID i-091dba2a9235236c4 IPv6 address - Hostname type - Public IPv4 address 54.147.186.243 open address Private IPv4 addresses 172.31.21.152 Instance state Running Public DNS ec2-54-147-186-243.compute-1.amazonaws.com open address Private IP DNS name (IPv4 only) -							

- Enter script in local and 'when = destroy'

```

1 resource "local_file" "test" {
2   filename = "pet.txt"
3   content = "i like cats"
4
5   provisioner "local-exec" {
6     when = destroy
7     command = "echo deleted the file called pet.txt > file.txt"
8   }
9 }

```

```

+ content_sha256      = (known after apply)
+ content_sha512      = (known after apply)
+ directory_permission = "0777"
+ file_permission     = "0777"
+ filename            = "pet.txt"
+ id                  = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.test: Creating...
local_file.test: Creation complete after 0s [id=48f7f126ff9d754918b417f4feb61c1a1448db89]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\ArjunandM\OneDrive\Desktop\Terraform_basics>

```

Tent

Terraform taint and untaint:

=====

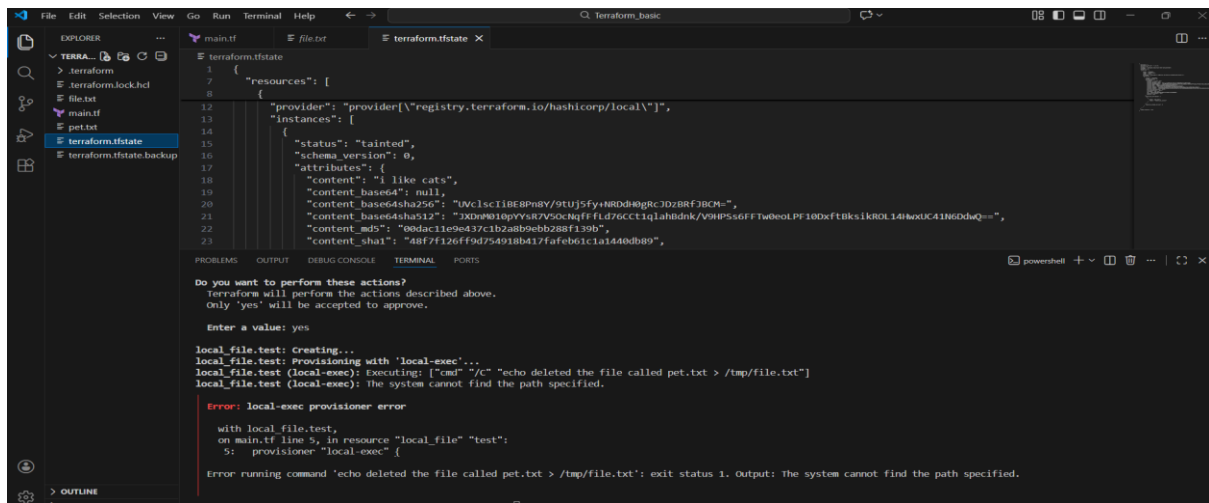
There would be cases when resource creation will get failed, if this happens then terraform will be marked as "Tainted".

We can see this when we execute terraform plan command and this will be replaced when we use terraform apply command.

If for any reason you have installed manually in EC2 and you want that to be replaced in next apply

then we can manually mark that resource as tainted in terraform so that it will be replaced next time when we use apply.

terraform taint aws_instance.webserver
to undo the changes we can use untaint command.



Terraform state commands:

=====

Terraform show --> To show the resources of the state file

Terraform state list --> List resources in the state

Terraform mv --> Move an item in the state

Terraform pull --> Pull current state and output to stdout

Terraform push --> Update remote state from a local state file

Terraform replace-provider --> Replace provider in the state

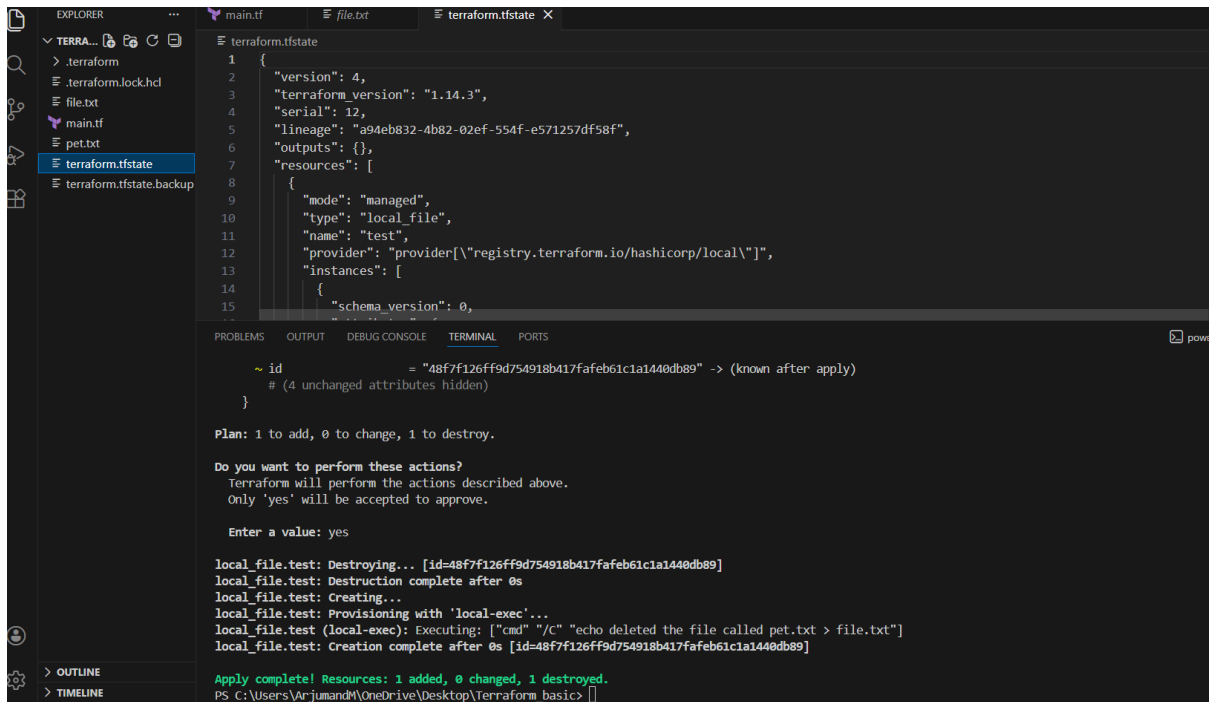
Terraform rm --> Remove instances from the state

Terraform show --> Show a resource in the state

Terraform Provisioners:

=====

Terraform provisioners allow us to execute command, scripts on remote machines or local place where terraform is installed



Debugging:

=====

terraform apply will provide us the logs/cause of the issue, but still if we want to dig deeper then we need to export a variable

export TF_LOG=TRACE

Set-Item -Path env:TF_LOG -value "TRACE"

Terraform provides 5 levels of logs:

- 1) INFO
- 2) WARNING
- 3) ERROR
- 4) DEBUG
- 5) TRACE

To store the logs permanently then we can export a path

Terraform import:

=====

Terraform import is used to import the existing infrastructure in terraform state file.

Once import is done then we can be able to create/delete and manage the infrastructure.

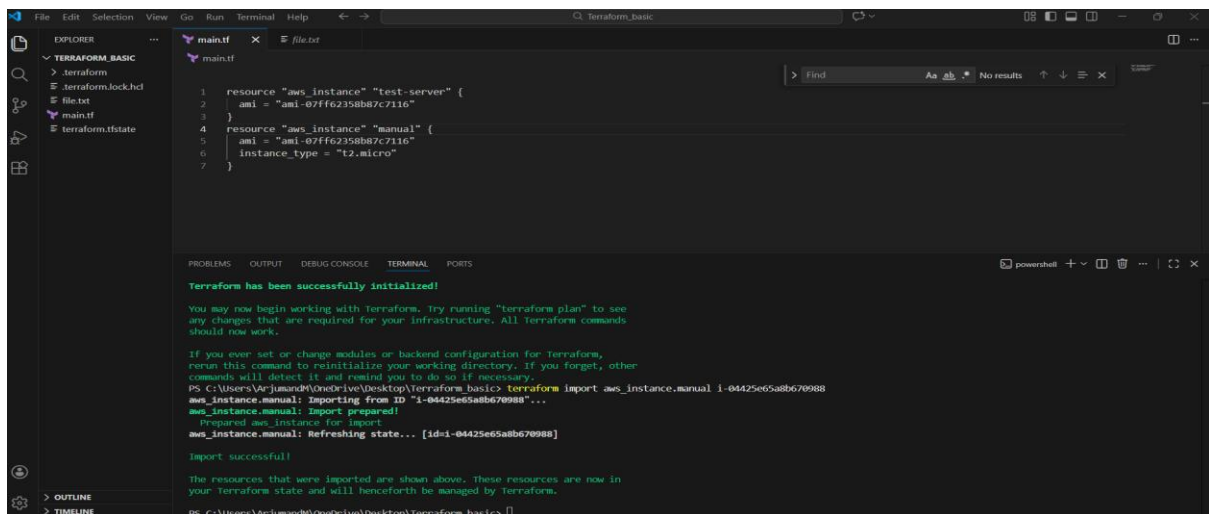
In order to import any resource we need to write the resource details in configuration file.

terraform import aws_instance.<name> instance_id

—>terraform import aws_instance.manual
i-0f1c4c02bc16389b9
—> i-0f1c4c02bc16389b9 is the ami id of other instance

Terraform import

—>terraform import aws_instance.manual
—>the ami id of other instance



3. Create one EC2 instance with httpd

installed using a Terraform script.

—> step 1: first add aws configure in your terraform folder
—> now In your terraform folder add main.tf file
—> and add below code
—> provide the ami = amazon machine image details

Terraform created instance with httpd:

```
resource "aws_instance" "test-server" {  
  
  ami = "ami-07ff62358b87c7116"  
  
  instance_type = "t2.micro"
```

```
key_name = "arj"
```

```
user_data = <<-EOF
```

```
#!/bin/bash
```

```
Sudo yum update -y
```

```
Sudo yum install httpd -y
```

```
Sudo systemctl start httpd
```

```
Sudo systemctl enable httpd
```

```
echo "<h1>Hello from Terraform EC2</h1>" > /var/www/html/index.html
```

```
EOF
```

```
tags = {
```

```
name = "First_serverT"
```

```
}
```

```
provisioner "local-exec" {
```

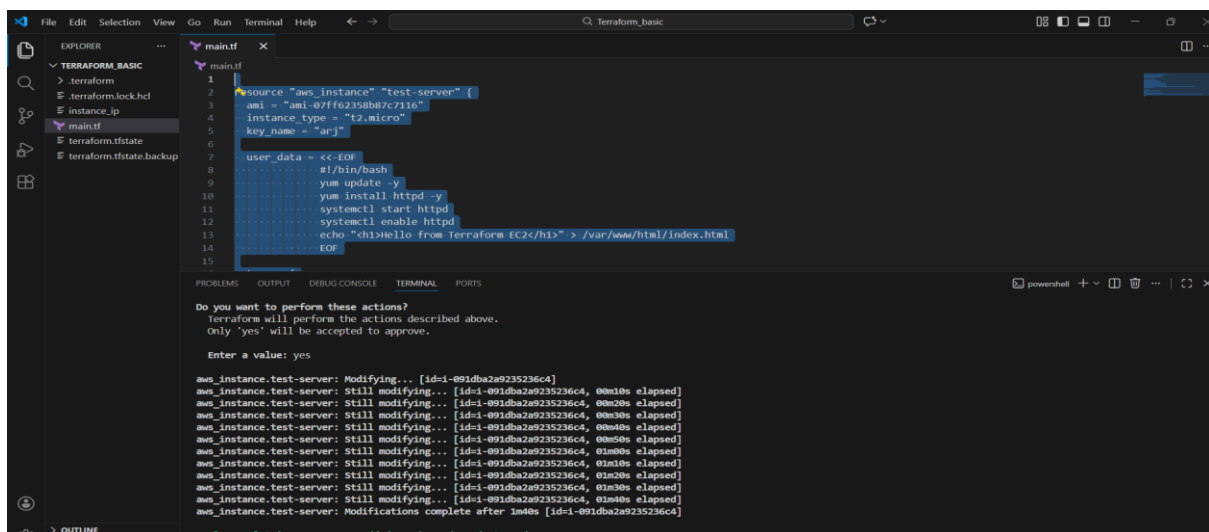
```
command = "echo instance ${aws_instance.test-server.public_ip} created! > instance_ip"
```

```
}
```

```
}
```

—> provide the ami = amazon machine image details

—> key_name = your pem key name



The screenshot shows a VS Code editor with a file named `main.tf` open. The file contains a Terraform configuration for an AWS EC2 instance. The configuration includes a `source` block for the `aws_instance` provider, followed by variables for `ami`, `instance_type`, and `key_name`. A `user_data` block is also present, containing a shell script that updates the system, installs `httpd`, starts and enables the service, and creates a file `/var/www/html/index.html` with the content `<h1>Hello from Terraform EC2</h1>`. The `tags` block is set to `name = "First_serverT"`. The `provisioner "local-exec"` block is used to execute a command that echoes the instance's public IP to a file named `instance_ip`.

The terminal output shows the execution of the Terraform plan and apply commands. The output indicates that the `aws_instance.test-server` is being modified. The output shows the instance's public IP address and the command being executed. The output also shows the status of the instance, indicating that it is still modifying. The output ends with the message "Apply complete! Resources: 0 added, 1 changed, 0 destroyed".

—-> terraform apply

—-> check your aws instances

i-04425e65a8b670988

PublicIPs: 18.234.60.171 PrivateIPs: 172.31.22.184

—> we can see same data is shown in server

Edit user data

Info

Instance ID

 i-04425e65a8b670988

Current user data

User data currently associated with this instance.

```
#!/bin/bash
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd
sudo echo "<ch1>Hello from Terraform EC2</h1>" > /var/www/html/index.html
```

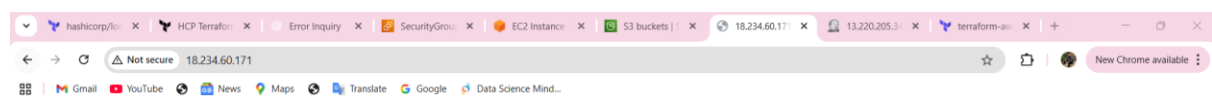
 Copy user data

 To edit your instance's user data you first need to stop your instance.

Cancel

Save

- HTTPD is launched using terraform



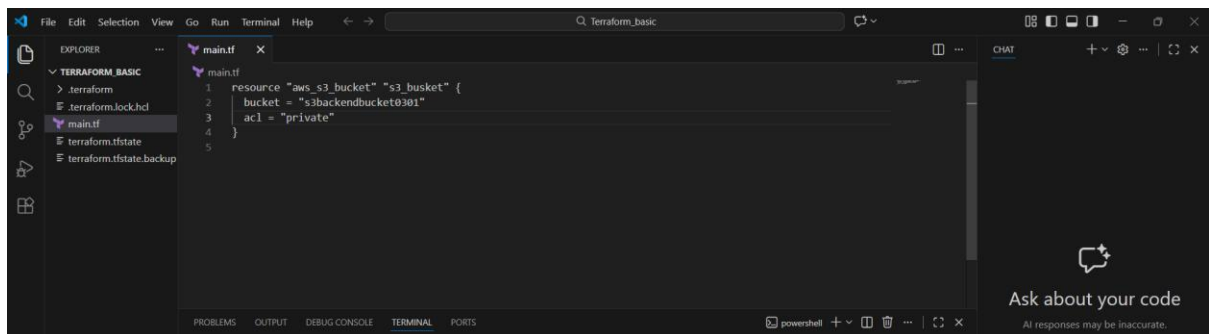
Hello from Terraform EC2

Task 4:

4. Set up S3 as backend for task

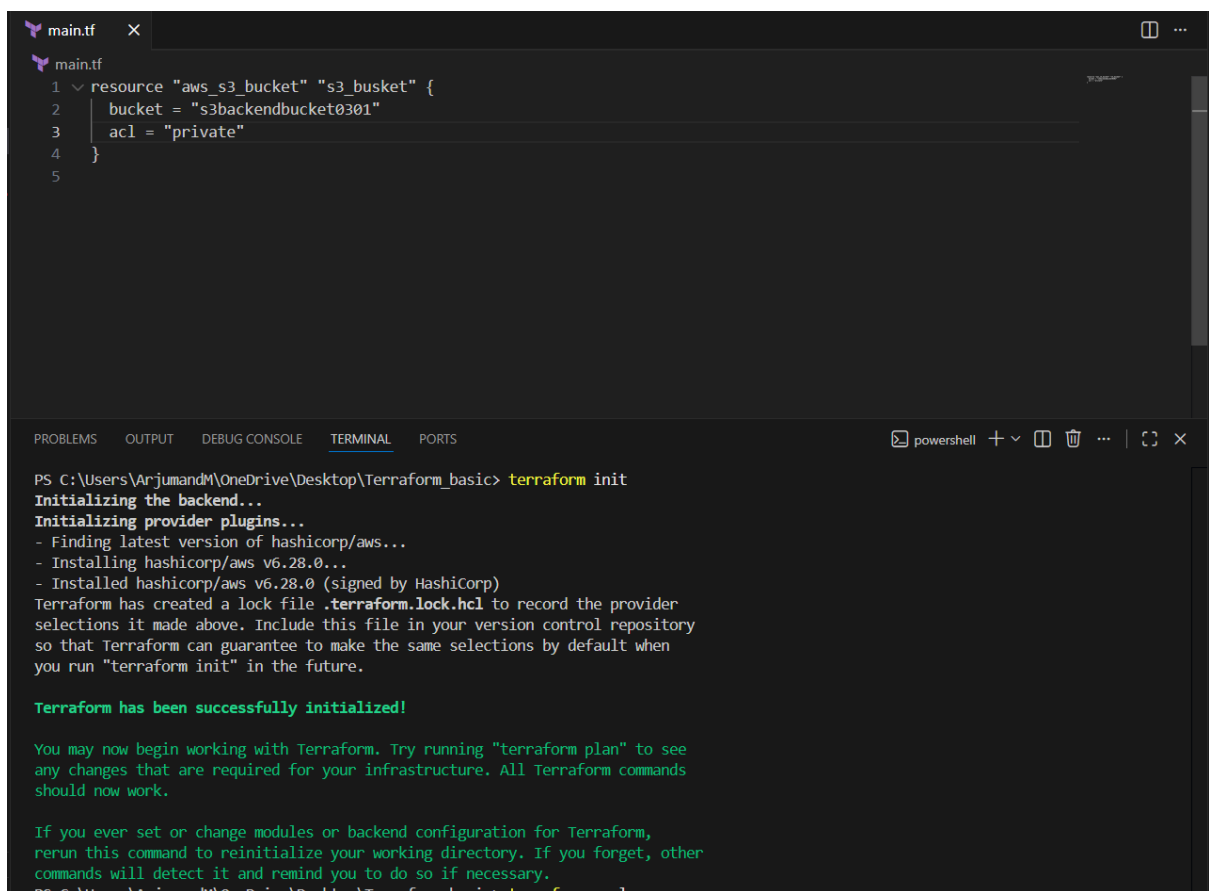
—>add your available bucket name

S3 bucket created from terraform



—> now apply

—> check your s3 bucket



```
acl is deprecated. Use the aws_s3_bucket_acl resource instead.

Error: creating S3 Bucket (s3backendbucket): operation error S3: CreateBucket, https response error StatusCode: 409, RequestID: 7T1TN1
XC1XW0EPwH, HostID: f2GAWQehhp6jIyLYK0wpvpK1UqMSWoolngg3hbcRJYa5F444z3lUXsRnPrbsduUbf9In/Vy2tFA=, BucketAlreadyExists: The requested buc
ket name is not available. The bucket namespace is shared by all users of the system. Please select a different name and try again.

with aws_s3_bucket.s3_bucket,
on main.tf line 1, in resource "aws_s3_bucket" "s3_bucket":
1: resource "aws_s3_bucket" "s3_bucket" {

PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  create
```

- Enter terraform apply

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_s3_bucket.s3_bucket: Creating...
aws_s3_bucket.s3_bucket: Creation complete after 6s [id=s3backendbucket0301]

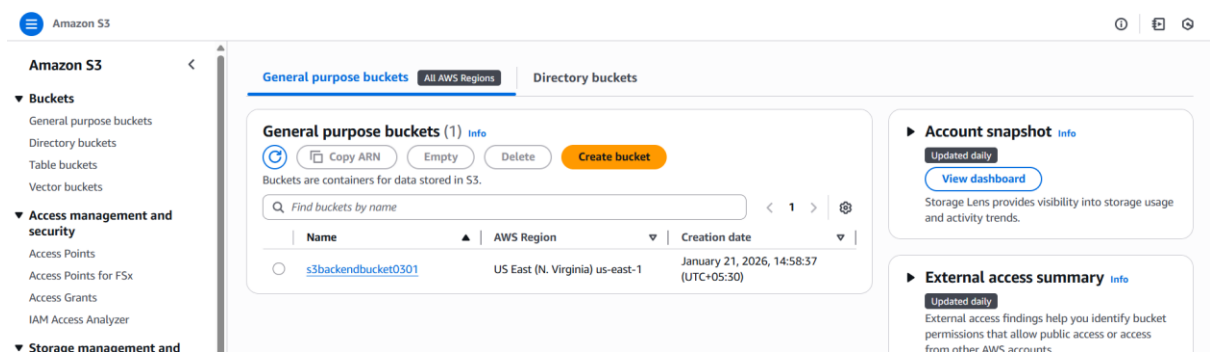
Warning: Argument is deprecated

with aws_s3_bucket.s3_bucket,
on main.tf line 3, in resource "aws_s3_bucket" "s3_bucket":
3:   acl = "private"

acl is deprecated. Use the aws_s3_bucket_acl resource instead.

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic>
```

- S3 bucket is created



Task 5:

5. Set up DynamoDB locking for task 3.

- Write a script for dynamodb in main.tf


```

5 resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {
6     name = "Terraform-state-lock-dynamo"
7     hash_key = "LockID"
8     read_capacity = 20
9     write_capacity = 20
10
11     attribute {
12         name = "LockID"
13         type = "S"
14     }
15 }

```

- Terraform init and terraform apply

```

with aws_s3_bucket.s3_bucket,
on main.tf line 3, in resource "aws_s3_bucket" "s3_bucket":
3:   acl = "private"

acl is deprecated. Use the aws_s3_bucket_acl resource instead.
(and one more similar warning elsewhere)

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_dynamodb_table.dynamodb-terraform-state-lock: Creating...
aws_dynamodb_table.dynamodb-terraform-state-lock: Still creating... [00m10s elapsed]
aws_dynamodb_table.dynamodb-terraform-state-lock: Creation complete after 15s [id=Terraform-state-lock-dynamo]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\Arjun\OneDrive\Desktop\Terraform-basics>

```

—> add the above details code

—> create one db and later add that data base in side the s3 backend

—> and apply

—> now check your dynamic db

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity
Terraform-state-lock-dynamo	Active	LockID (S)	-	0	0	Off	☆	Provisioned

Default Behaviour:

- By default, Terraform stores the state file (terraform.tfstate) locally inside your project folder.
- This state file is Terraform's memory of what resources exist in AWS.
- Problem: If you change laptop / multiple people run Terraform, state can mismatch and create duplicate resources.

Task 3: Create EC2 instance with httpd

installed

We created 1 EC2 instance using Terraform and installed Apache httpd using user_data.

What we did:

- Created EC2 instance (t3.micro)
- Used Amazon Linux AMI
- Added key pair for SSH login (key_name = "jenkins")
- Installed and started Apache automatically

User Data Script (Auto Install httpd)

- yum update -y
- yum install -y httpd
- systemctl start httpd
- systemctl enable httpd
- Created a webpage:

Hello from Terraform EC2 Apache Server

Output: When we opened EC2 public IP in browser, the page worked.

Task 4: Setup S3 as Backend (Remote State)

We configured Terraform to store the terraform.tfstate file in S3 instead of local system.

S3 Backend Details

- Bucket: waseem-s3-terraform-bucket02
- State file path (key): task3/terraform.tfstate
- Region: us-east-1

What this means:

- State is saved in S3
- Even if we change system, Terraform can still access the state
- Team members can use the same state safely

Task 5: Setup DynamoDB for Locking (State

Lock)

We created a DynamoDB table for state locking to prevent 2 users from running Terraform at the same time.

DynamoDB Details

- Table name: terraform-state-lock-dynamo
- Partition key: LockID (String)

What this does:

- Before Terraform apply, it creates a lock entry in DynamoDB
- So nobody else can apply changes at same time
- After apply completes, lock is removed automatically

Proof: We saw in terminal:

- Acquiring state lock
- Releasing state lock

And also we checked DynamoDB items showing LockID and Digest.

Final Result (What we achieved.